To start, import the necessary libraries. TensorFlow provides the core framework, and Keras offers the tools for building neural network layers.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

Step 2: Preparing the Dataset

For demonstration purposes, we use a synthetic dataset generated with the `make_regression` function. The dataset simulates a regression task where the target variable depends on input features.

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler # Create dataset
X, y = make_regression(n_samples=1000, n_features=10, noise=0.1) # Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Scale data for better performance
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 3: Defining the Model

Define a Sequential model with multiple Dense layers. Each Dense layer represents a fully connected layer in the neural network.

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)), # First hidden layer Dense(32, activation='relu'), # Second hidden layer Dense(1) # Output layer for regression ])
```

- **64, 32, and 1**: These numbers represent the number of neurons in each layer.
- **Activation Functions**: The ReLU activation function introduces non-linearity to help the model learn complex patterns.

Step 4: Compiling the Model

Before training, compile the model by specifying the optimizer, loss function, and metrics to monitor during training.

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```